

CAHIER DES CHARGES

CNC Energy Intelligence Platform

Prédiction et surveillance de la consommation énergétique des machines CNC

Champ	Valeur
Équipe	5 membres
Durée	3 semaines
Version	2.0 (révisée)
Date	Avril 2026

Table des matières

- 1. Présentation du projet
- 2. Périmètre fonctionnel
- 3. Architecture du système
- 4. Design de l'interface utilisateur
- 5. Workflow MLOps
- 6. Organisation de l'équipe
- 7. Estimation des coûts
- 8. Gestion des risques

1. Présentation du projet

1.1 Contexte

Dans un environnement de fabrication industrielle, les machines CNC constituent l'un des postes de consommation énergétique les plus importants d'un atelier. Leur utilisation intensive, combinée à l'absence de visibilité prédictive, expose les entreprises à des surcoûts énergétiques significatifs et à des pannes non anticipées. Face à la montée des obligations de reporting environnemental et à la pression croissante sur la maîtrise des charges opérationnelles, les industriels ont besoin d'outils intelligents capables de transformer des données brutes de capteurs en décisions concrètes.

1.2 Problématique

Les opérateurs et responsables de production n'ont aujourd'hui aucun moyen d'anticiper les pics de consommation énergétique de leurs machines CNC. Ils découvrent les anomalies après coup, sans possibilité d'identifier ni l'origine du problème ni la période concernée. Cette réactivité tardive génère trois types de pertes :

- des surcoûts énergétiques évitables ;
- une maintenance corrective plus coûteuse que préventive ;
- une incapacité à optimiser les plages horaires de production.

1.3 Objectif général

Concevoir et déployer une **plateforme intelligente de prédiction et de surveillance** de la consommation énergétique d'une machine CNC, capable de :

- fournir des prévisions à court terme basées sur un modèle supervisé ;
- détecter les comportements anormaux en quasi-temps-réel ;
- présenter ces informations via une interface claire et accessible à des profils non techniques.

Note de version : La version initiale (V1) repose sur un modèle supervisé (régression temporelle + détection d'anomalies statistique), privilégiant la fiabilité de livraison. L'intégration d'une boucle de Reinforcement Learning est positionnée comme évolution **V2** après validation du MVP.

2. Périmètre fonctionnel

2.1 Ce que le système fait (V1)

- Collecte les données issues des capteurs industriels ou d'un dataset de référence validé.
- Applique les transformations nécessaires pour rendre les données exploitables.
- Entraîne un **modèle supervisé** (ex. LightGBM, XGBoost ou Prophet) pour prédire la consommation énergétique à court terme.
- Détecte automatiquement les anomalies de consommation via des méthodes statistiques (ex. Isolation Forest, Z-score rolling).
- Intègre un mécanisme de validation humaine simple permettant à l'opérateur de confirmer ou corriger une prédiction.
- Expose l'ensemble de ces capacités via une API de service et un dashboard interactif.

2.2 Ce que le système ne fait pas

- Il ne pilote pas directement les machines.
- Il ne remplace pas un système SCADA ou un automate industriel.

- Il ne garantit pas une prédiction parfaite mais fournit une estimation probabiliste avec un niveau de confiance associé.
- Il ne couvre pas dans sa version initiale le cas de plusieurs machines simultanément, bien que l'architecture soit conçue pour l'extension.
- Il n'implémente **pas** de boucle de Reinforcement Learning en V1 (reporté en V2).

2.3 Dataset

■ Prérequis critique — à valider en Semaine 1.

Deux scénarios sont prévus :

Scénario	Source	Action
Données réelles disponibles	Capteurs CNC de l'environnement de déploiement	Utiliser directement après validation qualité
Données réelles non disponibles	Dataset public (ex. UCI, Kaggle — consommation énergétique industrielle)	Sélectionner et documenter le dataset en J1-J2

Aucun développement de pipeline ne démarre avant que le dataset soit confirmé.

3. Architecture du système

L'architecture est organisée en **cinq couches** aux responsabilités strictement séparées. Chaque couche communique avec les autres uniquement via des interfaces définies dès le début du projet.

3.1 Couche Ingestion — ETL

Responsabilité : pipeline ETL complet depuis la source de données jusqu'au stockage structuré.

- **Extraction :** collecte depuis les capteurs ou chargement depuis le dataset validé (format CSV/Parquet).
- **Transformation :** nettoyage, gestion des valeurs manquantes, normalisation des unités, enrichissement avec des métadonnées temporelles et opérationnelles.
- **Chargement :** stockage dans la Raw Zone, avec versioning via **DVC** pour garantir la reproductibilité.

3.2 Couche Stockage

Structure en trois zones aux responsabilités non chevauchantes :

Zone	Rôle
Raw Zone	Données sources post-ETL, conservées de manière immuable
Feature Store	Variables calculées avec version et timestamp, garantissant la reproductibilité
Model Registry	Géré par MLflow — historique complet des modèles (paramètres, métriques, artefacts)

3.3 Couche Traitement et Feature Engineering

Construction des variables représentatives à partir des données brutes. Trois catégories de transformations :

- **Transformations temporelles** : structure cyclique du temps (heure, jour, semaine) pour capturer les régularités de production.
- **Transformations historiques** : lags et fenêtres glissantes pour capturer les dynamiques à différentes échelles de temps.
- **Transformations d'interaction** : combinaisons de signaux révélant des relations non linéaires.

Le pipeline de feature engineering est **versionné, sérialisé et lié au modèle** auquel il correspond, garantissant que l'inférence applique exactement les mêmes transformations que l'entraînement.

3.4 Couche Machine Learning — Modèle Supervisé

┃ **Architecture V1 — simplifiée et livrable en 3 semaines.**

Modèle de prédiction : régression supervisée sur séries temporelles (LightGBM, XGBoost, ou Prophet selon les résultats d'expérimentation). Produit une estimation de consommation à court terme accompagnée d'un intervalle de confiance.

Modèle de détection d'anomalies : approche statistique (Isolation Forest ou Z-score rolling) comparant les observations courantes aux patterns historiques appris.

Boucle de validation humaine (simplifiée) :

Un opérateur ou responsable peut via le dashboard :

- confirmer qu'une anomalie détectée est réelle ;
- la rejeter si elle correspond à une opération connue (ex. changement d'outil planifié).

Ces retours sont enregistrés et constituent un jeu d'annotations pour une future amélioration du modèle.

Tous les runs d'entraînement sont tracés dans MLflow pour garantir l'auditabilité et permettre un retour vers une version précédente.

Roadmap V2 : intégration d'un agent RL dont la politique est optimisée sur la base des signaux de récompense enrichis par les annotations humaines accumulées en V1.

3.5 Couche Serving

Interface de service standardisée exposant les capacités du modèle :

- Soumettre des données récentes → obtenir une prédiction avec niveau de confiance.
- Consulter les anomalies détectées avec leur degré de sévérité.
- Vérifier l'état opérationnel du service (health check).

La couche serving est **conteneurisée** (Docker), déployable indépendamment de l'infrastructure sous-jacente. La documentation des contrats d'interface (OpenAPI/Swagger) est générée automatiquement.

3.6 Couche Monitoring

V1 : deux outils prioritaires. Les autres sont documentés mais non déployés.

Outil	Responsabilité	Statut V1
MLflow	Mémoire du système — historique des modèles, métriques, retour en arrière	✅ Déployé
Evidently	Détection de dérive des données en entrée — rapports lisibles par des non-spécialistes	✅ Déployé
NannyML	Performance estimée sans valeurs terrain — détection de concept drift	📄 Documenté (V2)
Grafana + Prometheus	Santé de l'infrastructure — disponibilité, latence, alertes opérationnelles	📄 Documenté (V2)

Déclencheurs de réentraînement automatique (V1) :

- Dérive des données détectée par Evidently.
- Dégradation confirmée dans MLflow une fois les valeurs réelles disponibles.

4. Design de l'interface utilisateur

4.1 Principes directeurs

L'interface est conçue pour deux profils simultanément :

- **L'opérateur machine** : clarté, alertes immédiates, actions en un clic.

- **Le responsable énergie** : tendances, indicateurs de performance, vue synthétique.

Design industriel moderne, lisible en conditions d'atelier. Hiérarchie visuelle guidant l'œil du statut global vers le détail.

4.2 Structure du dashboard

Zone supérieure — KPIs temps réel

Indicateurs clés dérivés des données ingérées : consommation courante, prévision à court terme, écart par rapport au comportement attendu. Unités et granularité déterminées lors de la phase de cadrage avec les utilisateurs finaux.

Zone centrale — Visualisation temporelle

Représentation superposant :

- la consommation historique mesurée ;
- la prédiction du modèle avec sa plage d'incertitude ;
- les points identifiés comme anormaux (marqueurs visuels distincts).

Une séparation visuelle claire distingue le passé observé du futur prédit.

Zone inférieure — Alertes et patterns

Divisée en deux parties :

- **Log d'alertes** : priorisées par sévérité, horodatées, rédigées en langage naturel, acquittables par l'opérateur.
- **Vue synthétique** : patterns de consommation sur période glissante pour identifier les régularités et anomalies récurrentes.

Module de validation humaine

Intégré au dashboard — permet de confirmer, corriger ou rejeter les prédictions et anomalies détectées. Les retours alimentent directement le jeu d'annotations du modèle.

4.3 Interactions clés

- Cliquer sur un point d'anomalie → panneau explicatif comparant les valeurs observées aux valeurs attendues.
 - Les KPIs sont cliquables et mènent à des vues détaillées.
 - Sur tablette (ligne de production) : mise en page simplifiée, graphique principal en plein écran, alertes accessibles en un geste.
-

5. Workflow MLOps

Le cycle de vie suit cinq phases en boucle continue.

Phase 1 — Expérimentation (Semaine 1)

Exploration dans un environnement dédié à l'analyse (notebook). Dès qu'une approche est validée, le code migre vers des modules structurés, testables et versionnés. **MLflow est intégré dès le premier run** pour qu'aucun résultat ne soit perdu.

Phase 2 — Entraînement

Pipeline reproductible enchaînant :

1. Chargement des données versionné via DVC.
2. Feature engineering via le pipeline sérialisé.
3. Entraînement du modèle supervisé avec optimisation des hyperparamètres.
4. Évaluation sur un jeu de test temporel strict (pas de data leakage).
5. Enregistrement conditionnel dans le Model Registry si les performances dépassent le seuil de référence.

Phase 3 — Validation humaine

Phase intercalée entre l'entraînement et le déploiement. L'équipe métier examine les prédictions du modèle candidat sur des cas réels, valide ou corrige les résultats. Ces annotations enrichissent le jeu de données d'évaluation.

Phase 4 — Déploiement

Production d'un artefact de service autonome et conteneurisé. Un pipeline d'intégration continue exécute les tests automatiquement avant tout déploiement et assure une mise en production sans interruption de service.

Phase 5 — Monitoring continu

- Génération de rapports de dérive réguliers via Evidently.
 - Comparaison des métriques en production aux métriques de référence dans MLflow.
 - Collecte des retours humains en continu via le module de validation.
 - Déclenchement d'un réentraînement automatique ou d'une alerte selon les seuils prédéfinis.
-

6. Organisation de l'équipe

6.1 Cinq modules isolés avec interfaces définies

Action obligatoire en J1 : session de cadrage d'une demi-journée pour figer les contrats d'interface entre modules (schémas JSON des endpoints API, format des features, structure du Feature Store). Chaque module travaille ensuite avec des mocks jusqu'à réception des livrables réels.

Module	Responsabilité et livrable
Data Pipeline	ETL complet, versioning des données via DVC, Feature Store. Livrable : datasets transformés, versionnés et documentés.
ML	Conception et entraînement du modèle supervisé, détection d'anomalies, enregistrement dans MLflow. Livrable : modèle versionné, évalué et documenté.
MLOps	Pipeline de réentraînement automatisé, intégration continue, monitoring via MLflow et Evidently. Livrable : système de déploiement et de surveillance automatisé.
Backend	API de service exposant les capacités du modèle, logique de détection d'anomalies, collecte des retours humains. Livrable : API documentée (OpenAPI), testée et consommable.
Frontend	Dashboard interactif, visualisations, module de validation humaine, intégration Backend. Livrable : interface déployable, responsive et validée.

6.2 Rythme de travail

- **Stand-up quotidien :** 10 minutes maximum — blocages, avancement, dépendances.
- **Revue de mi-projet (J+10) :** démonstration du MVP et recalibrage des priorités si nécessaire.
- **MVP livrable à J+15 :** prédiction fonctionnelle + dashboard basique. Garantit une démonstration même si les couches avancées ne sont pas finalisées.

6.3 Planning indicatif

Semaine	Priorités
S1	Validation dataset · Définition des interfaces · ETL de base · Première expérimentation ML
S2	Feature engineering complet · Entraînement et évaluation du modèle · API de base · Dashboard V1
S3	Intégration des modules · Monitoring MLflow + Evidently · Tests · Démo finale

6.4 Utilisateurs finaux

- **Opérateur machine** : consulte le dashboard quotidiennement, réagit aux alertes, valide les prédictions.
 - **Responsable énergie et maintenance** : analyse les tendances, planifie les interventions.
 - **Directeur de production** : consulte périodiquement les indicateurs globaux de performance énergétique.
-

7. Estimation des coûts

7.1 Infrastructure

L'ensemble du stack logiciel est **open source** — le coût en licences est nul. Le développement et le déploiement se font entièrement en local sur les machines de l'équipe, sans recours à une infrastructure cloud. Les coûts d'infrastructure sont limités aux postes de travail disponibles et à leur consommation électrique durant la durée du projet.

7.2 Coût de développement

Poste	Montant
5 membres × 15 jours × 4 h/jour	300 heures
Taux horaire moyen (profil junior qualifié, marché marocain)	150 DH/h
Coût total de développement	45 000 DH

Ce montant couvre uniquement le temps de développement et exclut tout coût d'infrastructure ou de licence.

8. Gestion des risques

Risque	Probabilité	Impact	Mitigation
Dataset indisponible ou de mauvaise qualité	Élevée	Critique	Identifier un dataset public de repli dès J1 (UCI, Kaggle) — décision obligatoire avant J3
Retard sur un module amont (Data Pipeline, ML)	Moyenne	Élevé	Interfaces mockées dès J1 — les modules aval ne sont pas bloqués

Risque	Probabilité	Impact	Mitigation
Complexité du feature engineering sous-estimée	Moyenne	Moyen	Commencer par un ensemble minimal de features (lags + cycliques) et enrichir si le temps le permet
Intégration finale difficile	Faible	Élevé	Intégration progressive hebdomadaire — pas d'assemblage unique en S3
Scope creep (ajout de fonctionnalités)	Moyenne	Moyen	Toute nouvelle fonctionnalité doit être évaluée en stand-up et approuvée par l'ensemble de l'équipe

Annexe — Stack technique recommandé

Composant	Technologie
Langage principal	Python 3.10+
Modèle de prédiction	LightGBM / XGBoost / Prophet
Détection d'anomalies	Isolation Forest (scikit-learn)
Versioning données	DVC
Tracking expériences	MLflow
Monitoring dérive	Evidently
API	FastAPI
Dashboard	Streamlit
Conteneurisation	Docker / Docker Compose
Tests	pytest

Document révisé — Version 2.0 — Avril 2026 Modifications principales : remplacement du RL par un modèle supervisé en V1, réduction du monitoring à MLflow + Evidently, ajout d'un protocole de validation du dataset, introduction d'un planning agile avec MVP à J+15, et section dédiée à la gestion des risques.